

Parfait — contrat validé, on passe en mode exécution. Voici le plan d'intégration lecture, tâche par tâche, avec les critères de fin. Tout référence le contrat mergé ([docs/AKAL\\_Contrat\\_Donnees\\_v1.1.md](#)).

## ● IBRAHIM — Backend

### I-1. Correctifs schéma (feature branch **fix/schema-pre-migrations**) —

#### ⚠ à faire en premier

1. `Parcelle.type_culture` : ajouter le champ en enum fermée (choices : `agrumes`, `cereales`, `olivier`, `maraichage`, `arboriculture`, `vigne`), requis — contrat §3.1
2. `Conversation` : contrainte `UNIQUE(annonce, initiateur)` nommée `uniq_conversation_annonce_initiateur`, retirer `destinataire_id` — §3.7
3. `AgriScore.parcelle` : `OneToOne` → **ForeignKey**, ajouter `version_ponderation` (varchar) — §3.5
4. Retirer `contour` de `Parcelle` → nouveau modèle `DonneesGeo` (`OneToOne`, `PolygonField`) — §3.2
5. `Photo` : garder `ordre` (integer, unique par annonce, commence à 0), **supprimer** `is_principale` — §3.6
6. Créer `StatistiqueAnnonce` (annonce FK, date, vues) et retirer le compteur `vues` d'`Annonce` — §3.4
7. `makemigrations --name schema_corrections_contrat_v1_1` puis `migrate` en local

**Critère de fin** : migrations passent sur une base vierge + PR ouverte. Comme Baroud a l'échéance en cours, merge autorisé à J+5 sans réponse.

### I-2. drf-spectacular (~30 min, peut se faire avant I-1)

`pip install drf-spectacular`

Settings (`DEFAULT_SCHEMA_CLASS`), routes `/api/schema/` et `/api/schema/swagger-ui/`. **Critère de fin** : Swagger UI accessible en local.

### I-3. Endpoints lecture conformes au contrat §4

- `GET /api/annonces/` : serializer **liste allégée** (§4.4 — `photo_principale` = URL de la photo `ordre 0`, `score_courant.score_global` seul, jamais `DonneesGeo`)
- `GET /api/annonces/<slug>/` : serializer détail complet (sous-objet `parcelle`, `photos` triées par `ordre`, `score_courant` = `AgriScore` le plus récent ou `null`)

- `GET /api/geo/regions/` : référentiel non paginé [{code, nom}]
- Pagination `PageNumberPagination` : `page_size=12, max_page_size=50`
- Filtres query params : `region, type_culture, statut_foncier, acces_eau, prix_min/max, surface_min/max, ordering` (champs autorisés : `date_publication, prix_max, surface_ha`)
- **CORS** : `django-cors-headers` avec `http://localhost:3000` autorisé en dev — c'est le piège n°1 de la première connexion, à faire tout de suite
- URLs médias absolues via MinIO public (§4.7) — le front ne concatène rien

**Critère de fin** : les trois endpoints répondent dans Swagger avec des données de seed conformes à l'exemple JSON §4.4.

## I-4. Fixtures de seed

Une commande ou fixture avec ~15 annonces réalistes (régions variées, statuts variés, certaines sans AgriScore pour tester le cas `null`, une sans photo). **Critère de fin** : `python manage.py loaddata seed_annonces` (ou équivalent) peuple une base de démo.

# ● TOI (MÉGANE) — Frontend

## M-1. Fondations client API — démarrable maintenant, sans attendre Ibrahim

- `frontend/src/lib/api.ts` : client fetch centralisé — base URL depuis `NEXT_PUBLIC_API_URL`, helper d'erreurs unique (lit `detail` pour les erreurs simples, les clés de champ pour les 400 — contrat §4.6), typage générique des réponses paginées {count, next, previous, results}
- `frontend/src/types/parcelle.ts` : adapter — `id: string` (UUID), ajouter `slug`, sous-objet `parcelle`, `score_courant nullable`, `sous_scores` typé `Record<string, number>` (clés provisoires, jamais en dur — §3.5)
- `frontend/src/lib/mapAnnonceToParcelle.ts` : mapping DTO API → ton type front actuel
- `.env.local` : `NEXT_PUBLIC_API_URL=http://localhost:8000/api` et `NEXT_PUBLIC_USE MOCKS=true`
- Toggle mock/réel dans la couche data : `parcelles.ts` devient le fallback mock, branché derrière la variable d'env

**Critère de fin** : test unitaire de `mapAnnonceToParcelle()` qui passe contre l'exemple JSON copié-collé du §4.4 — pas contre l'API réelle.

## M-2. Branchement lecture (dès que I-3 est en local chez Ibrahim)

- Brancher liste (`/api/annonces/` + filtres via query params), détail (`/api/annonces/<slug>/`), régions (`/api/geo/regions/`)
- Les liens des cartes passent de l'id au **slug**
- Gérer les trois états : loading (tes Skeleton\*), erreur, `score_courant === null` → « Score en cours de calcul »
- Toute divergence contrat/réalité → **issue GitHub label `api-mismatch`**, jamais à l'oral

**Critère de fin** : catalogue + détail affichés depuis l'API en local, `NEXT_PUBLIC_USE MOCKS=false`.

### M-3. Pagination + filtres réels

- Pagination branchée sur `count/next/previous` (URLs absolues, pas de recalcul)
- `FiltresSidebar` : les valeurs des selects viennent de `/api/geo/regions/` et des enums du contrat, persistance des filtres dans l'URL

**Critère de fin** : filtrage + navigation entre pages sans rechargement complet, URL partageable.

## COMMUN

### C-1. Première connexion en binôme (~1 h, dès I-3 + M-1 prêts)

Ibrahim lance son back en local avec le seed, toi ton front pointé dessus. Vous déroulez ensemble : liste → filtre → détail → cas score null → cas sans photo. Chaque écart = issue `api-mismatch` séance tenante. **Critère de fin** : zéro écart ouvert, ou tous tracés en issues.

### C-2. Démo intégration lecture

Parcours complet démontré, capture ou vidéo courte → c'est ta preuve de jalon pour le plan directeur (J1 décalé au 19/07 — vous êtes dans les temps si C-1 se fait cette semaine).

## Ordre d'exécution

Aujourd'hui → toi : M-1 | Ibrahim : I-2 puis I-1  
 J+1 / J+2 → toi : M-1 fin | Ibrahim : I-3 + I-4  
 Dès I-3 dispo → toi : M-2  
 Ensuite → M-3, puis C-1 ensemble, puis C-2  
 J+5 (si silence) → Ibrahim merge I-1 (protocole)

Le seul couplage dur est M-2 ← I-3. Tout le reste avance en parallèle. Ensuite viendra l'avenant écriture (auth, favoris, dépôt, messagerie — 6a à 6e), mais n'ouvre pas ce chantier tant que C-2 n'est pas démontré : la lecture de bout en bout est ton jalon J1.

